

函数

Modelica语法详解

苏州同元软控信息技术有限公司版权所有，
未经授权不得转载、复制或传播或以其他方式使用，
侵权必究

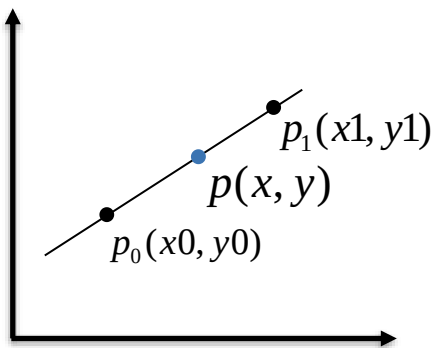
苏州同元软控信息技术有限公司

2024年2月27日



TONGYUAN
Software and Control

Example



已知 p_0 、 p_1 点的横纵坐标，如何根据 p 点的横坐标求纵坐标？

函数

$$y(x, x_0, y_0, x_1, y_1) = x \frac{y_1 - y_0}{x_1 - x_0} - \frac{x_1 + x_0}{2} \frac{y_1 - y_0}{x_1 - x_0} + \frac{y_1 + y_0}{2}$$

Modelica代码

```
function LineWithProtected
  input Real x;
  input Real p0[2];
  input Real p1[2];
  output Real y;
  protected
    Real m = (p1[2] - p0[2]) / (p1[1] - p0[1]);
    Real b = (p1[2] + p0[2] - m * (p1[1] + p0[1])) / 2.0;
  algorithm
    y := m * x + b;
end LineWithProtected;
```

那么如何继续计算多项式？ ...

Example

$$y(x, \vec{c}) = \sum_{i=1}^N c_i x^{N-i} = c_1 x^{N-1} + c_2 x^{N-2} + \dots + c_{N-1} x + c_N$$

其中: x 为独立变量, $\vec{c}$ 代表系数组成的向量

例如4阶多项式的表达式为:

$$y(x, \vec{c}) = c_1 x^3 + c_2 x^2 + c_3 x + c_4$$
$$= (((c_1 x + c_2) x) + c_3) x + c_4$$

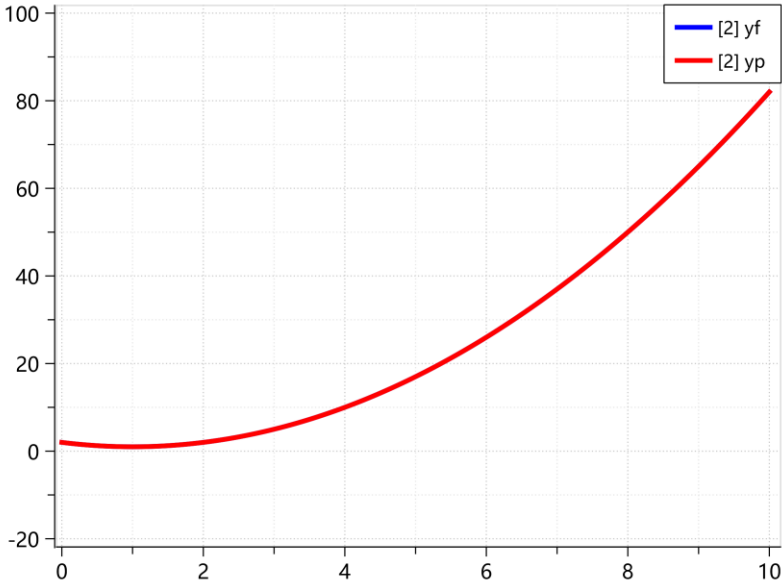
使用递归的方式更加高效, 仅用加法和乘法, 避免了处理更复杂的求幂运算。

```
function Polynomial "多项式计算"  
  input Real x "独立变量";  
  input Real c[:] "多项式系数";  
  output Real y "多项式的值";  
  protected  
    Integer n = size(c, 1);  
  algorithm  
    y := c[1];  
    for i in 2:n loop  
      y := y * x + c[i];  
    end for;  
  end Polynomial;
```

函数声明

```
model EvaluationTest1  
  Real yf;  
  Real yp;  
  equation  
    yf = Polynomial(time, {1, -2, 2});  
    yp = time ^ 2 - 2 * time + 2;  
  end EvaluationTest1;
```

函数调用



如何在Modelica中实现递归?
如何增加算法的重用性?

苏州同元软控信息技术有限公司版权所有, 侵权必究。



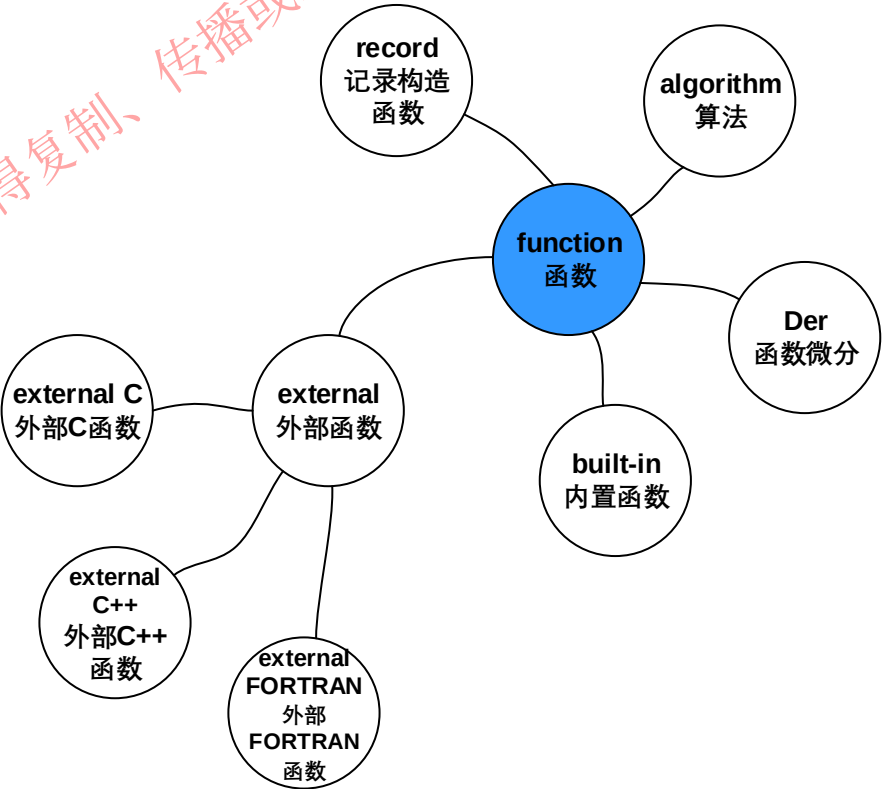
思考

函数是数学意义上的纯函数，也就是说相同的输入总是具有**相同**的输出，并且调用顺序与调用次数**不改变**所在模型的仿真状态。

```
function Polynomial "多项式计算"  
  input Real x "独立变量";  
  input Real c[:] "多项式系数";  
  output Real y "多项式的值";  
protected  
  Integer n = size(c, 1);  
algorithm  
  y := c[1];  
  for i in 2:n loop  
    y := y * x + c[i];  
  end for;  
end Polynomial;
```

```
model EvaluationTest1  
  Real yf;  
  Real yp;  
equation  
  yf = Polynomial(time, {1, -2, 2});  
  yp = time ^ 2 - 2 * time + 2;  
end EvaluationTest1;
```

函数该如何声明?
由哪些部分组成?
怎么调用函数?



苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；



目录

1. 函数声明

2. 函数调用

3. 内置函数

4. 记录构造函数

5. 函数微分注解声明

6. 本章回顾

1. 函数声明

Modelica函数是使用关键字function的特化类，一般由输入变量、输出变量、中间变量、算法组成。

函数一般结构

```
function name
  input Type1 in1;
  input Type2 in2;
  input Type3 in3 := default_expr1 "Comment" annotation(...);
  ...
  output TypeO1 out1;
  output TypeO2 out2 := default_expr2;
  ...
protected
  <local variables>
  ...
algorithm
  ...
  <statements>
  ...
end name;
```

```
function LineWithProtected "计算直线点坐标"
  input Real x "输入横坐标";
  input Real p0[2] "直线上p0点坐标";
  input Real p1[2] "直线上p1点坐标";
  output Real y "对应x的y坐标";
protected
  Real m = (p1[2] - p0[2]) / (p1[1] - p0[1]);
  Real b = (p1[2] + p0[2] -
    m * (p1[1] + p0[1])) / 2.0 ;
algorithm
  y := m * x + b;
end LineWithProtected;
```

定义function名称

- 定义输入
- 定义输出
- 定义中间变量

描述function算法

结束function定义



1. 函数声明

注意事项

```
function IsVectorEqual "判断向量是否相等"
input Real v1[:];
input Real v2[:];
input Real eps(min = 0) = 0;
output Boolean result;
protected
Integer i = 1;
algorithm
result := false;
if size(v1, 1) <> size(v2, 1) then
return;
end if;
result := true;
while i < size(v1, 1) loop
if abs(v1[i] - v2[i]) > eps then
result := false;
return;
end if;
i := i + 1;
end while;
end IsVectorEqual;
```

1. 函数中所有数组的长度必须可以通过输入形参或者函数中的参数、常量确定。
2. 函数中输入形参赋值只是给输入形参一个缺省值，即调用函数时如果不给eps赋值，eps默认为0。输入形参是只读的，在算法中不能给输入形参赋值
3. public区域的变量声明是函数的形参，必须有“input”或“output”前缀，protected区域的变量声明是函数的临时变量，不能有“input”和“output”前缀。。
4. 函数中不能用于连接，不能有“equation”，至多有一个“algorithm”区域或外部函数接口。
5. 函数中不能调用der、initial、terminal、sample、pre、edge、change、delay、cardinality、reinit等内置操作符和函数，也不能使用when语句。
6. “return”语句表示退出函数调用，返回值取输出形参的当前值。

1. 函数声明

变量定义与记录类配合使用：

```
record Complex "复数"  
  Real re "实数部分";  
  Real im "虚数部分";  
end Complex;
```

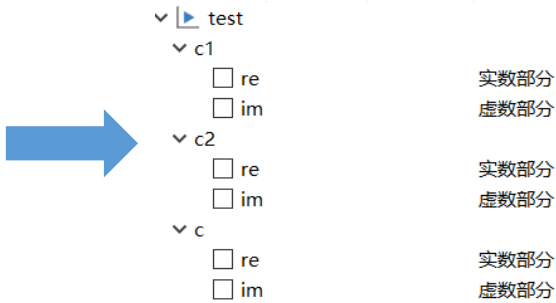
方式1：

```
function Complex_add  
  input Complex u;  
  input Complex v;  
  output Complex w(  
    re = u.re + v.re,  
    im = u.im + v.im);  
end Complex_add;
```

```
model test  
  Complex c1(re = 2, im = 3);  
  Complex c2(re = sin(time), im = cos(time));  
  Complex c;  
equation  
  c = Complex_add(c1, c2);  
end test;
```

方式2：

```
function Complex_add  
  input Complex u;  
  input Complex v;  
  output Complex w;  
algorithm  
  w := Complex(  
    re = u.re + v.re,  
    im = u.im + v.im);  
end Complex_add;
```

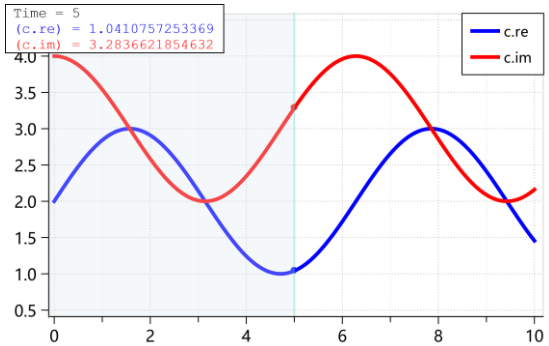


优点：

- record相当于集合，能够增加形参的重用性，有效减少工作量
- 将相关变量使用record集合化，可增加仿真结果的直观性。

方式3：

```
function Complex_add  
  input Complex u;  
  input Complex v;  
  output Complex w;  
algorithm  
  w.re := u.re + v.re;  
  w.im := u.im + v.im;  
end Complex_add;
```



目录

1. 函数声明

2. 函数调用

3. 内置函数

4. 记录构造函数

5. 函数微分注解声明

6. 本章回顾

2. 函数调用-形参输入

```
function Equal
  input Real x1;
  input Real x2 = 2;
  input Real x3 = 3;
  output Real y1;
  output Real y2;
  output Real y3;
algorithm
  y1 := x1;
  y2 := x2;
  y3 := x3;
end Equal;
```

```
model Equal_Test
  parameter Real a = 1;
  parameter Real b = 2;
  parameter Real c = 3;
  Real y1;
  Real y2;
  Real y3;
equation
  (y1,y2,y3) = Equal(a, b, c);
end Equal_Test;
```

方式1：按位置传参

```
(y1,y2,y3) = Equal(a, b, c);
```

方式2：按形参名字传参 (推荐使用)

```
(y1,y2,y3) = Equal(x3 = c, x2= b, x1 = a);
```

方式3：混合方式传参 (不推荐使用)

```
(y1,y2,y3) = Equal(a, x3 = c);
```

结果等价

名字	值	单位
Equal_Test	T=0	
☐ a	1	
☐ b	2	
☐ c	3	
☐ y1	1	
☐ y2	2	
☐ y3	3	

注意：

- 1. 实参与形参数据类型与维度均保持一致
- 2. 形参如有缺省值则可以不传参，实参默认为缺省值。
- 3. 混合传参时，按形参名字传参的实参必须放在按位置传参的实参之后，不推荐使用。



2. 函数调用-形参输出

```
function Equal
  input Real x1;
  input Real x2 = 2;
  input Real x3 = 3;
  output Real y1;
  output Real y2;
  output Real y3;
algorithm
  y1 := x1;
  y2 := x2;
  y3 := x3;
end Equal;
```

```
model Equal_Test
  parameter Real a = 1;
  parameter Real b = 2;
  parameter Real c = 3;
  Real y1;
  Real y2;
  Real y3;
equation
  (y1,y2,y3) = Equal(a, b, c);
end Equal_Test;
```

全部传参

(y1,y2,y3) = Equal(a, b, c);

求解前两个值

(y1,y2) = Equal(x3 = c, x2= b, x1 = a);

求解第1,3两个值

(y1,,y3) = Equal(a, x3 = c);

求解后两个值

(,y2,y3) = Equal(x3 = c, x2= b, x1 = a);

注意:

- 多个输出使用 “()” 表示
- 输出传参只能按照位置传参，与function中定义的输出形参顺序一致。

y1 = 1
y2 = 2
y3 = 3

y1 = 1
y2 = 2

y1 = 1
y3 = 3

y2 = 2
y3 = 3

未赋值变量需要使用
其他等式进行赋值

2. 函数调用

向量化调用

Modelica的函数向量化调用，即返回标量值的函数可以应用向量化调用方式实现以数组作为实参调用函数，Modelica自动以数组每个元素作为参数来调用函数，返回结果数组，极大地提高了建模效率。

内置函数

```
sin({a, b, c}) = {sin(a), sin(b), sin(c)}
sin([a,b,c]) = [sin(a),sin(b),sin(c)]
sin([1,2;3,4]) =[sin(1), sin(2); sin(3), sin(4)]
div({a,b,c},{d,e,f}) = {div(a,d),div(b,e), div(c,f)}
```

自定义函数

```
function vec_sum
  input Real A[:];
  output Real res;
algorithm
  res := 0;
  for i in 1:size(A,1) loop
    res := res + A[i];
  end for;
end vec_sum;
```

```
model vec_sum_Test
  parameter Real a[:,:] = {{1, 2, 3, 4}, {2, 4, 6, 8}};
  Real sum[size(a, 1)];
equation
  sum = vec_sum(a);
end vec_sum_Test;
```

```
sum = {vec_sum({1, 2, 3, 4}), vec_sum({2, 4, 6, 8})};
```

名字	值
vec_sum_Test	T=0
a	
sum	
sum[1]	10
sum[2]	20

等价于



目录

1. 函数声明

2. 函数调用

3. 内置函数

4. 记录构造函数

5. 函数微分注解声明

6. 本章回顾

3. 内置函数

分类	函数名称	说明
数值函数	abs(a)	返回标量绝对值
	sign(a)	返回标量符号
	sqrt(a)	返回标量平方根
转换函数	Integer(e)	返回枚举型序数
	String(a, <options>)	将一个标量表达式转为字符串表示
事件触发数学函数	div(x,y)	返回x/y的商且丢弃小数部分
	mod(x,y)	返回x/y的整数模, 即 $x - \text{floor}(x/y) * y$
	rem(x,y)	返回x/y整除的余数
	ceil(x)	返回不小于x的最小整数
	floor(x)	返回不大于x的最大整数
	integer(x)	返回不大于x的最大整数, 结果必为整型
	der(expr)	返回表达式对时间求导
求导和特殊用途函数	delay()	返回表达式的迟滞信号
	semiLinear(x, positiveSlope, negativeSlope)	若 $x \geq 0$ 结果为 $\text{positiveSlope} * x$, 否则结果为 $\text{negativeSlope} * x$

分类	函数名称	说明
数学函数	sin(x)	正弦函数
	cos(x)	余弦函数
	tan(x)	正切函数
	asin(x)	反正弦函数
	acos(x)	反余弦函数
	atan(x)	反正切函数
	atan2(x,y)	四象限反向切值
	sinh(x)	双曲正弦函数
	cosh(x)	双曲余弦函数
	tanh(x)	双曲正切函数
	exp(x)	自然指数函数
	log(x)	自然对数(e 为底), $x > 0$
	log10(x)	10为底的对数, $x > 0$

关于数组函数详细内容在《数组》章节中进行详细讲解
关于事件相关函数详细内容在《事件》章节中进行详细讲解



3. 内置函数

数值函数

- **abs(a)**: 返回标量绝对值
- **sign(a)**: 返回标量符号
- **sqrt(a)**: 返回平方根

```
model Value
  parameter Real a = -4;
  Real b;
  Real c;
  Real d;
  equation
    b = abs(a);
    c = sign(a);
    d = sqrt(abs(a));
  end Value;
```

b = 4;
c = -1;
d = 2;

注意:

- sqrt(a)中的输入实参必须为**非负数**
- abs(a)、sign(a)、sqrt(a)中的输入实参类型均为**实型或整型**;

3. 内置函数

转换函数

- Integer(e): 返回枚举值的序数

```
model Integer_f
type SelectMode = enumeration(
  Mode1 "模式1",
  Mode2 "模式2",
  Mode3 "模式3")
"选择模式";
parameter SelectMode m = SelectMode.Mode2;
Real b;
equation
  b = Integer(m);
end Integer_f;
```

b = 2;

- 注意:**
- Integer(e)输入实参为枚举型中确定的枚举值;
 - 返回值为整型或实型;

- String(a, <options>): 将一个标量表达式转为字符串表示

```
model String_f
parameter Real a1 = 3.1415926;
String b1;
equation
  b1 = String(a1, significantDigits = 2,
  minimumLength = 5, leftJustified = false);
end String_f;
```

b1 = " 3.1" ;

- 说明:**
- string()输入实参的数据类型可为实型、整型、布尔型、枚举型。
- significantDigits = 2, 表示字符串有效数字个数为2, 赋值必须为整型, 且只能用于实型输入实参。
 - minimumLength = 5, 表示字符串最小长度, 用空格填充未用空间, 赋值必须为整型, 可用于所有类型输入实参。
 - leftJustified = false, 表示右对齐, 如果赋值为true则为左对齐, 赋值必须为布尔型, 可用于所有类型输入实参。



3. 内置函数

事件触发数学函数

在when子句之外使用，当返回值不连续时，将触发状态事件

- **div(x,y)**: 返回x/y的商，舍弃小数部分
- **mod(x,y)**: 返回x/y的整数模，即 $x - \text{floor}(x/y) * y$
- **rem(x,y)**: 返回x/y的整余数
- **ceil(x)**: 返回不小于x的最小整数
- **floor(x)**: 返回不大于x的最大整数
- **integer(x)**: 返回不大于x的最大整数，结果必为整型

注意:

1. div(x,y)、mod(x,y)、rem(x,y)中x、y均为整型时，输出实参可以为整型或者实型，否则输出实参均为实型。
2. ceil(x)、floor(x)参数和输出只可为实型，integer(x)参数只可为实型，输出只可为整型。

```
model Event_math
  parameter Real x = 3.2;
  parameter Real y = 2;
  Real a;
  Real b;
  Real c;
  Real d;
  Real e;
  Integer f;
equation
  a = div(x, y);
  b = mod(x, y);
  c = rem(x, y);
  d = ceil(x);
  e = floor(x);
  f = integer(x);
end Event_math;
```



```
a = 1;
b = 1.2;
c = 1.2;
d = 4;
e = 3;
f = 3;
```



3. 内置函数

数学函数

函数名称	说明
sin(x)	正弦函数
cos(x)	余弦函数
tan(x)	正切函数, $x \neq \pi/2, 3\pi/2, \dots$
asin(x)	反正弦函数, $-1 \leq x \leq 1$
acos(x)	反余弦函数, $-1 \leq x \leq 1$
atan(x)	反正切函数
atan2(x,y)	四象限反向切值
sinh(x)	双曲正弦函数
cosh(x)	双曲余弦函数
tanh(x)	双曲正切函数
exp(x)	自然指数函数
log(x)	自然对数(e 为底), $x > 0$
log10(x)	10为底的对数, $x > 0$

注意:

- 1. 以上函数参数为实型或整型, 输出为实型。

```
model Math
parameter Integer a = 2;
Real b;
Real c;
Real d;
equation
b = sin(a);
c = cos(a);
d = tan(a);
end Math;
```



```
b = sin(2);
c = cos(2);
d = tan(2);
```

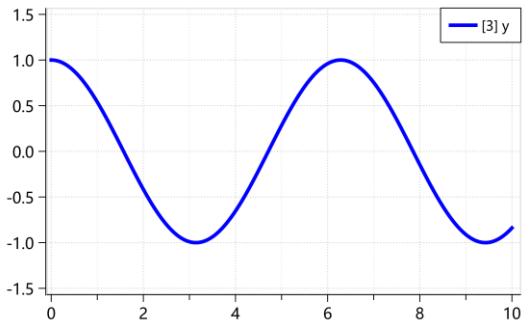


3. 内置函数

求导和特殊用途函数

- **der (expr):** 返回表达式对时间求导

```
model Der
  Real x = sin(time);
  Real y;
  equation
    y = der(x);
end Der;
```



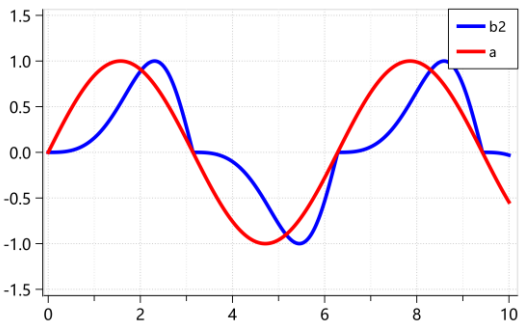
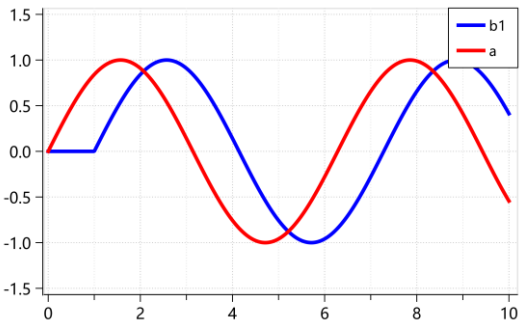
注意:

1. 参数必须可微分。

- **delay (expr, delayTime):**
- **delay (expr, delayTime, delayMax)**

} 返回表达式的迟滞信号

```
model Delay
  parameter Real delaytime = 1;
  Real a = sin(time);
  Real b1;
  Real b2;
  equation
    b1 = delay(a, delaytime);
    b2 = delay(a, sin(time), delaytime);
end Delay;
```



注意:

1. delay (expr, delayTime)时, delayTime必须为参数或常数。
2. delay(expr, delayTime, delayMax)时, delayMax必须为参数或常数, 此时delayTime可为变量, delayTime ≤ delayMax。

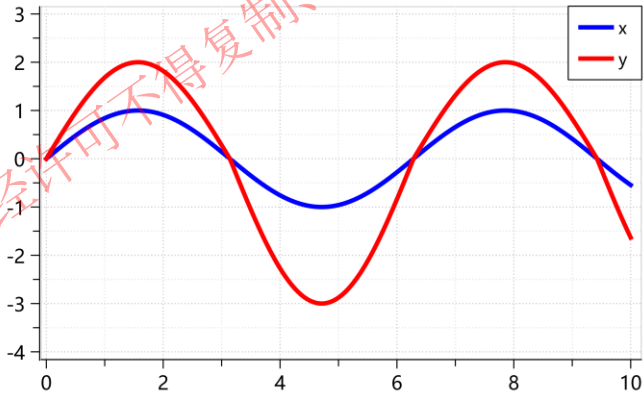


3. 内置函数

求导和特殊用途函数

- semiLinear(x, positiveSlope, negativeSlope):** 若 $x \geq 0$ 结果为 $\text{positiveSlope} * x$, 否则结果为 $\text{negativeSlope} * x$

```
model SemiLinear
  Real x = sin(time);
  Real sa = 2;
  Real sb = 3;
  Real y;
equation
  y = semiLinear(x, sa, sb);
end SemiLinear;
```



说明:

semiLinear函数用于处理流体系统中的回流，例如：

$$H_flow = semiLinear(m_flow, port.h, h);$$

即，焓流速H_flow可以依据流向，由物质流速m_flow和返侵比焓来计算。



目录

1. 函数声明

2. 函数调用

3. 内置函数

4. 记录构造函数

5. 函数微分注解声明

6. 本章回顾

4. 记录构造函数

每实例化一个记录，都隐含定义了一个记录构造函数，它与记录同名，与记录类有相同的作用域，记录类赋值方式与函数相同。

```
package demo
record Record1
  parameter Real r0 = 0;
end Record1;
record Record2
  extends Record1;
  constant Real c1 = 2.0;
  constant Real c2;
  parameter Integer n1 = 5;
  parameter Integer n2;
  parameter Real r1 "comment";
  parameter Real r2 = sin(c1);
  parameter Real r4;
  parameter Real r5 = 5.0;
  Real r6[n1];
  Real r7[n2];
  final parameter Real r3 = cos(r2);
end Record2;
end demo;
```

隐式定义
➡

```
package Demo
function Record1
  input Real r0 = 0;
  output Record1 'result'(r0 = r0);
end Record1;
function Record2
  input Real r0 = 0;
  input Real c1 = 2;
  input Real c2;
  input Integer n1:=5;
  input Integer n2;
  input Real r1 "comment";
  input Real r2:=sin(c1);
  input Real r4;
  input Real r5 = 5.0;
  input Real r6[n1];
  input Real r7[n2];
  output Record2 'result'
  (r0 = r0, c2 = c2, n1 = n1, n2 = n2, r1 = r1, r2 = r2, r4 = r4,
  r5 = r5, r6 = r6, r7 = r7);
  protected
  final parameter Real r3 = cos(r2);
end Record2;
end Demo;
```

```
model Record_test
  Record2 r1 = Record2(r0 = 1, c1 = 4, c2 = 2, n1 = 2, n2 = 3, r1 = 1,
    r2 = 2, r4 = 5, r5 = 5, r6 = {1, 2}, r7 = {1, 2, 3});
  Record2 r2 = Record2(1, 4, 2, 2, 3, 1, 2, 5, 5, {1, 2}, {1, 2, 3});
end Record_test;
```

记录类赋值说明：

- 1. 前缀为：final parameter时，被记为保护组件，不能进行重新赋值。
- 2. 前缀为constant、parameter或无前缀时，原记录类中赋值为缺省赋值，可以重新进行赋值。

传参方式：

- 按形参名传参(推荐)
- 按位置传参



目录

1. 函数声明

2. 函数调用

3. 内置函数

4. 记录构造函数

5. 函数微分注解声明

6. 本章回顾

5. 函数微分注解声明

对于内置函数，可以直接使用der对时间进行求导；
对于自定义函数，可以使用smoothOrder让工具自行推导出导函数

```
model der_func_test
function sin_cos
  input Real x;
  output Real y;
algorithm
  y := 0;
  y := sin(x) * cos(x);
end sin_cos;
Real x = sin(time) * cos(time);
Real y = der(x);
Real x2 = sin_cos(time);
Real y2 = der(x2);
end der_func_test;
```



形式:

function + 函数名称

<属性描述>

annotation(smoothOrder = n)

<行为描述>

end + 函数名称

```
function sin_cos
  input Real x;
  output Real y;
  annotation(smoothOrder = 1);
algorithm
  y := 0;
  y := sin(x) * cos(x);
end sin_cos;
```



5. 函数微分注解声明

在一些特定情况下，smoothOrder无法推出导函数，还可以自己推导导函数并进行相关绑定。

如何关联原函数和推导的导函数？

```
model der_func_test
function sin_cos
  input Real x;
  output Real y;
algorithm
  y := 0;
  y := sin(x) * cos(x);
end sin_cos;
Real x = sin(time) * cos(time);
Real y = der(x);
Real x2 = sin_cos(time);
Real y2 = der(x2);
end der_func_test;
```



形式:

function + 函数名称

<属性描述>

annotation(derivative=导函数路径)

<行为描述>

end + 函数名称

```
function sin_cos
  input Real x;
  output Real y;
  annotation(derivative = sin_cos_d);
algorithm
  y := sin(x) * cos(x);
end sin_cos;
```

```
function sin_cos_d
  input Real x;
  input Real der_x;
  output Real der_y;
algorithm
  der_y := der_x * cos(x) ^ 2
          - der_x * sin(x) ^ 2;
end sin_cos_d;
```



5. 函数微分注解声明

多阶复合函数求导

$$y = f(x), x = h(t)$$

```
function f
  input Real x;
  output Real y;
  annotation (derivative = der_f);
algorithm
  ...
end f;
```

$$y' = \frac{dy}{dx} \cdot \frac{dx}{dt}$$

```
function der_f
  input Real x;
  input Real der_x;
  output Real der_y;
  annotation (derivative(order = 2) = der_2_f);
algorithm
  ...
end der_f;
```

系统自行求出

$$y'' = \frac{dy^2}{d^2x} \cdot \left(\frac{dx}{dt}\right)^2 + \frac{dy}{dx} \cdot \frac{dx^2}{d^2t}$$

```
function der_2_f
  input Real x;
  input Real der_x;
  input Real der_2_x;
  input Real der_y;
  output Real der_2_y;
algorithm
  ...
end der_2_f;
```

引用第一阶结果

```
Model der_F
  Real x = sin(time);
  Real y;
  Real y1;
  Real y2;
equation
  y = f(x);
  y1 = der(y);
  y2 = der(y1);
end sin_cos;
```

形式:

function + 函数名称

<属性描述>

annotation(derivative (order = n)=n阶导函数路径)

<行为描述>

end + 函数名称

说明:

order = n 表示n阶导函数，n为整型，且缺省值为1;

5. 函数微分注解声明

示例

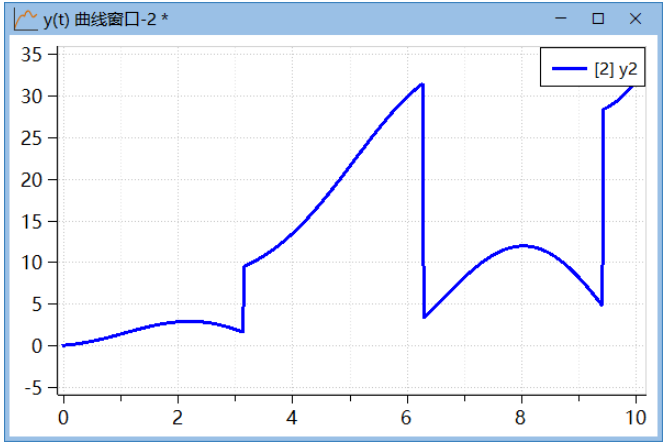
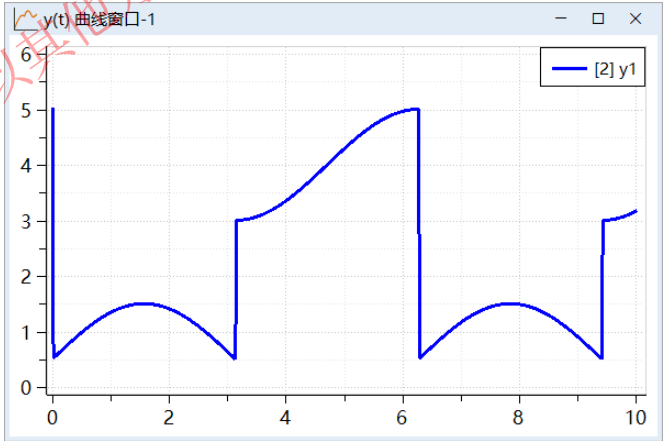
$$y_1 = \begin{cases} \sin(x) + 0.5 & \sin(x) > 0 \\ \cos(x) + 4 & \sin(x) \leq 0 \end{cases}$$
$$y_2 = x \cdot y_1$$

- 建一个model，以time作为函数的输入
1. 求输出y1, y2
 2. 求解y2一阶导数和二阶导数的值

```
function func_y1
  input Real x;
  output Real y1;
algorithm
  if sin(x) > 0 then
    y1 := sin(x) + 0.5;
  else
    y1 := cos(x) + 4;
  end if;
end func_y1;
```

```
function func_y2
  input Real x;
  output Real y2;
protected
  Real y1 = func_y1(x);
algorithm
  y2 := x * y1;
end func_y2;
```

```
model Cal
  Real x = time;
  Real y1;
  Real y2;
equation
  y1 = func_y1(x);
  y2 = func_y2(x);
end Dy2;
```



5. 函数微分注解声明

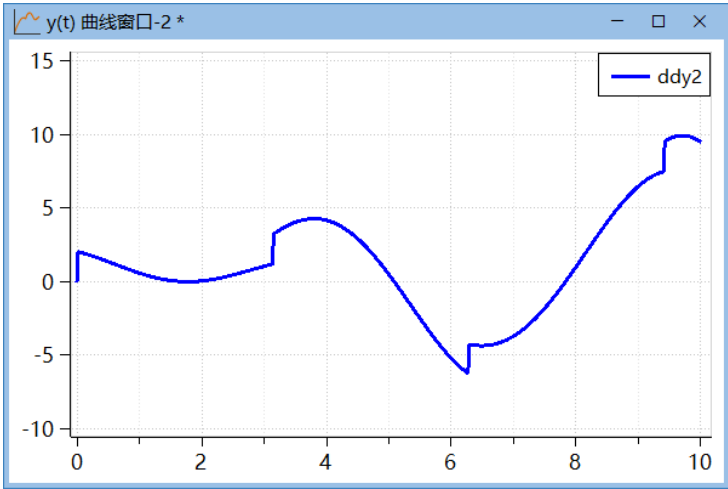
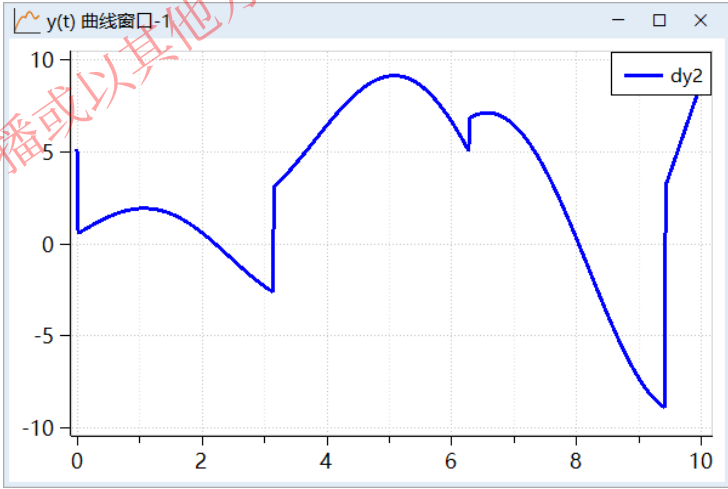
2. 求解y2一阶导数和二阶导数的值

```
function func_y2
  input Real x;
  output Real y2;
  annotation (derivative = Der1);
protected
  Real y1 = func_y1(x);
algorithm
  y2 := x * y1;
end func_y2;
```

```
function Der1 "y2一阶导数"
  input Real x;
  input Real der_x;
  output Real dy2;
  annotation (derivative(order = 2) = Der2);
algorithm
  if sin(x) > 0 then
    dy2 := (0.5 + sin(x) + x * cos(x)) * der_x;
  else
    dy2 := (cos(x) - x * sin(x) + 4) * der_x;
  end if;
end Der1;
```

```
function Der2 "y2二阶导数"
  input Real x;
  input Real der_x;
  input Real dy2;
  output Real ddy2;
algorithm
  if sin(x) > 0 then
    ddy2 := (2 * cos(x) - x * sin(x)) * der_x;
  else
    ddy2 := (-2 * sin(x) - x * cos(x)) * der_x;
  end if;
end Der2;
```

```
model DDy2 "求y2二阶导数"
  Real x = time;
  Real y1;
  Real y2;
  Real dy2;
  Real ddy2;
equation
  y1 = func_y1(x);
  y2 = func_y2(x);
  dy2 = der(y2);
  ddy2 = der(dy2);
end DDy2;
```



目录

1. 函数声明

2. 函数调用

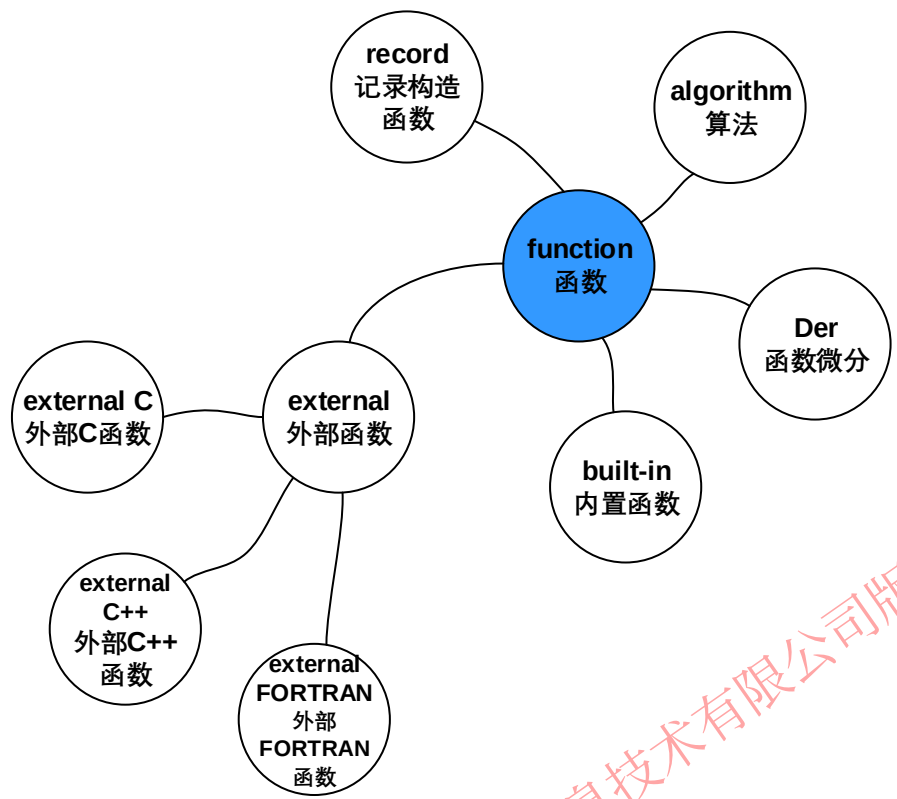
3. 内置函数

4. 记录构造函数

5. 函数微分注解声明

6. 本章回顾

6. 本章回顾



- 1. 函数由输入变量，输出变量，算法区域构成。
- 2. 函数调用时的三种传参方式：按位置传参、按名称传参、缺省传参。
- 3. 各个内置函数的使用。
- 4. 记录类中final parameter前缀的变量不能更改赋值。
- 5. 通过函数中添加注解derivative 关联导函数。
- 6. MWorks.Sysplorer 可调用C、C++ 和 FORTRAN77，并三种调用方式：调用头文件、调用链接库文件、无外部文件调用（后续课程《MWORKS.Sysplorer外部接口》章节讲授）。

苏州同元软控信息技术有限公司版权所有，未经许可不得复制或传播，或以其它方式使用；



6. 本章回顾

课堂回顾

1. 定义函数的关键词是 () 。
A. function B. equation C. algorithm D. block
2. 下列不属于内置函数的是 () 。
A. div B. sin C. mod D. vec_sum
3. sign(-5)的结果是 () 。
A. 1 B. 0 C. -1 D. -5
4. floor(4.5)的结果是 () 。
A. 4 B. 4.5 C. 5 D. 5.0
5. div(5,2)的结果是 () 。
A. 1 B. 2 C. 3 D. 4
6. 下面属于事件触发数学函数的是 () 。
A. abs B. div C. sin D. pre
7. 下面返回结果是整型的函数是 () 。
A. rem B. ceil C. floor D. integer
8. 关联导函数关键词是 () 。
9. 求两个数的模的函数是 () 。
10. rem(5,2)的结果是 () 。

6. 本章回顾

课后作业

$$1. y_1 = \begin{cases} \sin(x) + 0.5 & \sin(x) > 0 \\ \cos(x) + 4 & \sin(x) \leq 0 \end{cases}$$

$$y_2 = x \cdot y_1$$

建一个model, 以time作为函数的输入, 求输出y1, y2。

2. 在第1题的基础上求解y2一阶导数和二阶导数的值。
3. 猴子第一天采摘了若干个桃子, 第二天, 吃剩下的桃子的一半, 还不过瘾, 又多吃了一个; 以后每天都吃前一天剩下的一半多一个, 到第10天想再吃时, 只剩下一个桃子了。问第一天共采摘了多少个桃子?
4. 如果第15天发现只剩下1个桃子, 那么第一天共采摘了多少个桃子?
5. 复现所有内置函数的实例(可以改变参数或对实例进行改进)

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

谢谢！